Count Inversions

AlgoExpert · Very Hard · Sorting

The problem

- Number of pairs i < j with a[i] > a[j], counted during merge sort.

Intuition

- Merge sort counts inversions: when a right element is placed before remaining left elements, they all form inversions.

Approach

- 1. Sort halves recursively
- 2. During merge, add remaining left count when taking from the right
- 3. Sum

Java solution

```java
public class CountInversions {
    public static int countInversions(int[] array) {
        int[] a = array.clone();
        return (int) count(a, new int[a.length], 0, a.length - 1);
    }

    private static long count(int[] a, int[] aux, int lo, int hi) {
        if (lo >= hi) return 0;
        int mid = (lo + hi) >>> 1;
        long inv = count(a, aux, lo, mid) + count(a, aux, mid + 1, hi);
        System.arraycopy(a, lo, aux, lo, hi - lo + 1);
        int i = lo, j = mid + 1;
        for (int k = lo; k <= hi; k++) {
            if (i > mid) a[k] = aux[j++];
            else if (j > hi) a[k] = aux[i++];
            else if (aux[j] < aux[i]) { a[k] = aux[j++]; inv += mid - i + 1; }
            else a[k] = aux[i++];
        }
        return inv;
    }
}
```

Complexity

- Time: O(n log n)
- Space: O(n)

Recap

- Merge sort counts inversions: when a right element is placed before remaining left elements, they all form inversions.
- Complexity: O(n log n)